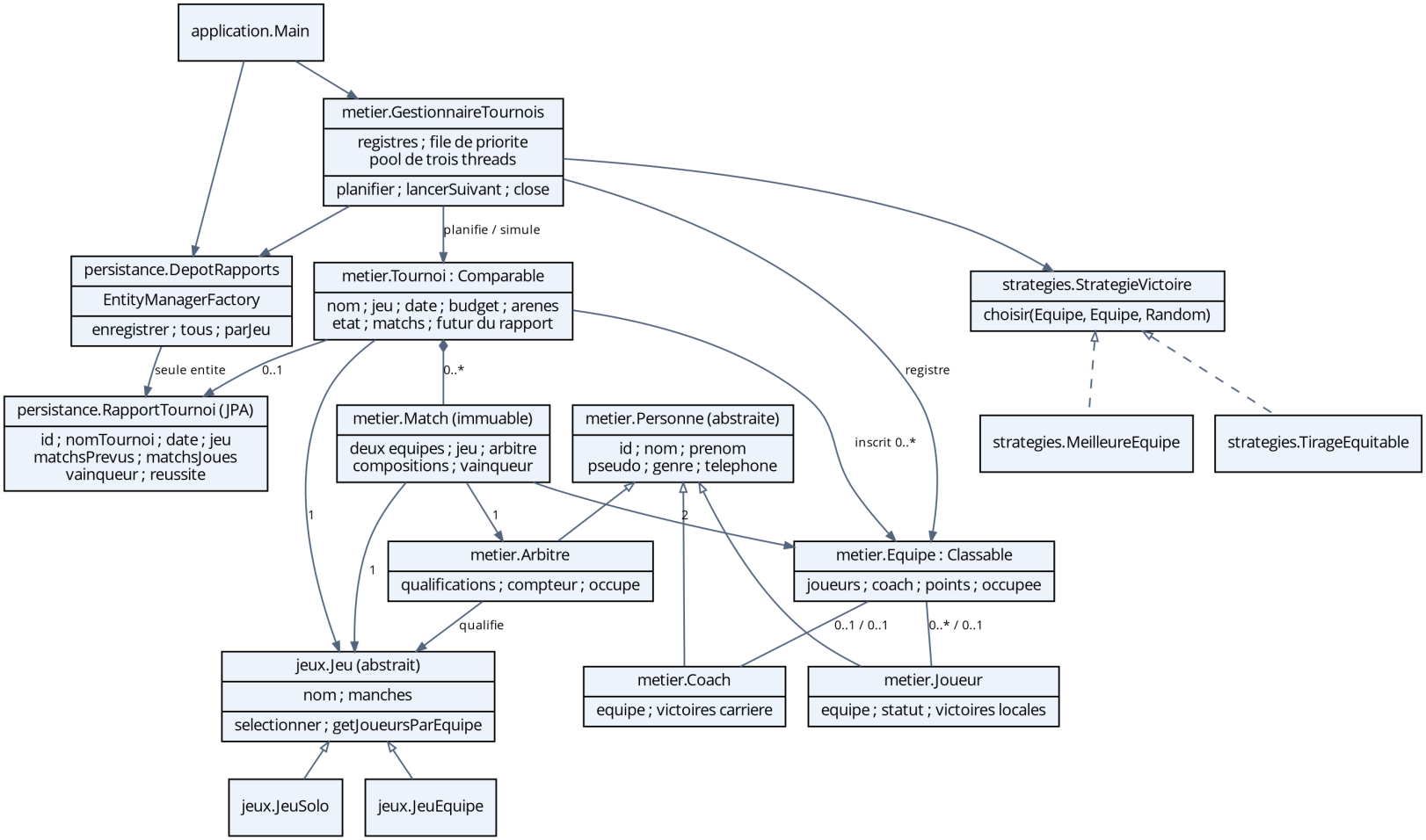
IMT Arena — conception et organisation



Lecture. Triangle vide : héritage ; trait pointillé : réalisation d'interface ; losange plein : composition. Les préfixes correspondent aux dossiers Java. Le diagramme Mermaid fournit aussi les interfaces, énumérations, Verrou et Textes.
Packages. application : démonstration ; metier : participants, équipes, tournois et coordination ; jeux : sélection ; strategies : victoire ; persistance : rapports et JPA ; util : textes. Les classes métier restent ensemble afin de conserver les modifications internes sans ajouter de setters publics.
Règles. Un joueur ou coach ne rejoint qu'une équipe ; ajouter un membre déjà ailleurs réalise un transfert si les équipes sont libres. Trois victoires depuis l'arrivée rendent le joueur professionnel. Le coach garde son total de carrière. Une victoire rapporte trois points ; un incident ne rapporte rien. Les rapports sont les seuls objets persistés.

Collections, Strategy et synchronisation

Besoin / structure	Justification
Registres : HashMap	Accès moyen O(1) par nom ou identifiant ; doublons refusés à l'enregistrement.
Planification : PriorityQueue	Tournoi Comparable par date puis nom immuables. Tête O(1), ajout/retrait O(log n). La liste affichée est une copie triée.
Effectifs et résultats : ArrayList	Ordre conservé et parcours simple. Les listes internes sont protégées par des copies non modifiables.
Qualifications : HashSet	Absence de doublon ; recherche moyenne O(1). Les jeux restent des classes extensibles.
Classements : List et Comparator	Tri O(n log n), points ou budget décroissants puis nom. Liste recréée à chaque consultation, sous verrou ; pas d'arbre conservé sur des scores modifiables.
Groupes : LinkedHashMap	Une équipe gagnante associée à ses matchs. Ordre de première victoire conservé. Listes copiées et map non modifiable.

Strategy. StrategieVictoire définit choisir. TirageEquitable utilise le hasard ; MeilleureEquipe compare les points puis le nom. Modifier l'objet stratégie change la règle sans modifier Match. Aucun autre pattern n'est ajouté sans besoin.

Concurrence. Un ExecutorService contient trois threads, une tâche FutureTask par tournoi. Le compteur enCours contrôle aussi les admissions. Verrou.OBJET protège transferts, réservations et publication finale. Les pauses, stratégies et attentes du futur sont hors verrou. Les résultats sont calculés localement, puis points, rapport, état et libérations sont publiés à la fin. La sauvegarde JPA reste sous verrou pour simplifier la cohérence, au prix d'un ralentissement possible des autres opérations protégées.

Temps et hasard. Chaque tournoi possède son Random et sa graine. Deux boucles i puis j=i+1 créent toutes les paires. Cadence : temps par match / arènes, sans thread de match. Durée cible M*t/A, hors surcoûts ; les arènes sont un débit logique, pas un planning sportif. Un arbitre disponible est réservé à chaque tournoi. Réussite : 10*joués >= 7*prévus. Vainqueur : points locaux puis nom ; « Aucun » si zéro match joué.

Persistance et arrêt. RapportTournoi est une entité JPA ; Hibernate enregistre dans H2 fichier. Un EntityManager par opération et une transaction par sauvegarde. Une erreur remonte via Future.get, sans garder les réservations. close ferme le pool et attend la fin ; une interruption demande l'arrêt des simulations. Les extensions doivent coopérer. Le gestionnaire ferme avant le dépôt. Les tests vérifient les règles, la concurrence et la relecture après réouverture.